Take-Home Assignment: Orders and Settlements

Estimated time: 6–8 hours
Role: Full Stack Developer (3–6 years experience)

Overview

Build a small web application where users create orders with line items, record payments against those orders (full or partial), and view a dashboard with order status and amounts due.

This exercise reflects common patterns in B2B and SaaS products: documents with line items, partial payments, and derived status. You do **not** need invoicing or accounting experience to complete it.

Requirements

1. Authentication

- Implement sign up and log in (email + password is sufficient).
- Each user must only see and modify their own data.

2. Orders

Users can create orders with:

Field	Description
Customer	Customer name (plain string is fine)
Due date	When payment is expected
Line items	See below

Each **line item** has:

Field	Description
Description	Item name
Quantity	Number (≥ 1)
Unit price	Price per unit

The system must auto-compute:

- **Subtotal** — sum of (quantity × unit price) across all lines.
- **Order total** — same as subtotal for this assignment (no order-level tax or discount required).

3. Order status

Derive status from payments and due date:

Status	Condition
`pending`	No payments recorded
`partially_paid`	Some payment recorded, but less than order total
`paid`	Total payments equal order total

`overdue`	Past due date and not fully paid

Document any edge cases (e.g. an order that was overdue but is now fully paid) in your README.

4. Payments

Users can record a payment against an order:

Field	Description
Amount	Payment amount (≥ 0.01)
Date	Date payment was made
Note	Optional free text

Rules:

- Total payments for an order must **never exceed** the order total.
- Multiple payments on the same order are allowed.
- Reject over-payment with a clear, actionable error (e.g. include the maximum allowed amount).

5. Dashboard

Build a dashboard that shows:

- List of orders with: customer, status, order total, amount paid, amount due, due date.
- Filter by status.
- Order detail page: line items and full payment history.

6. API

Expose a REST API.

- CRUD for orders.
- Endpoint to record payments with server-side validation.
- Consistent error responses with helpful messages.

Document whether orders remain editable after the first payment, or become read-only — either approach is acceptable if explained.

Sample scenario

Use this flow to verify your implementation:

1. Create an order: 2 × $500 = **$1,000 total**, due in 7 days.
2. Record payment of **$400** → status should be `partially_paid`, amount due **$600**.
3. Record payment of **$600** → status should be `paid`, amount due **$0**.
4. Attempt to record another **$1** payment → should be rejected with a clear error.

Stretch goals (optional)

- **Refunds** — record a refund (negative payment or separate refund entity).
- **Audit log** — track status changes with timestamps.
- **Export** — download orders as CSV for a date range.

Technical guidelines

- **Stack:** Your choice.

- **Deployment:** Required — include a live URL to a deployed version of the app.
- **Tests:** Tests for payment allocation, status transitions, and over-payment rejection are appreciated.
- **Concurrency:** Consider what happens if two payments are submitted at the same time — document your approach even if you don't implement full locking.

Deliverables

Submit a Git repository containing:

1. **Source code** — backend, frontend, and any migrations/seed scripts.
2. **Deployed live URL** — a publicly accessible link to the running app.
3. **README** with:

- Prerequisites and step-by-step setup.
- API overview (main endpoints).
- Status derivation rules and edge-case decisions.
- Assumptions and tradeoffs.
- What you would improve before production.
- The deployed URL (also include it in your submission email).

Optional: a short Loom/video walkthrough (5–10 minutes).

What we evaluate

Area	What we look for
Correctness	Line item math, payment totals, and status logic
Business rules	Over-payment prevention, partial payments, overdue handling
API design	Clear endpoints, validation errors with resolution hints
Frontend	Dashboard usability, order detail, payment flow
Code quality	Readable structure, transactional writes where appropriate
Communication	README clarity and documented decisions

Questions?

If anything is ambiguous, make a reasonable assumption, document it in your README, and proceed.

Good luck — we look forward to reviewing your work.